Agentic QA Automation Pipeline — Solution Overview

Date: 2026-06-26 **Author:** Umair Iftikhar **Status:** Ready for evaluation

---

1. What the solution does

The Agentic QA Automation Pipeline converts a user story (URL + acceptance criteria) into runnable Playwright test scripts, executes them in a visible browser, and produces a structured execution report — all from a single web UI, with **no paid API required**.

It eliminates the manual effort of:

- Translating acceptance criteria into Playwright test code
- Maintaining locators across UI changes
- Writing repetitive execution summaries

A QA engineer pastes a URL + story into the UI and ends up with:

- A formal user story file
- Page Object Model classes (one per page)
- A spec file per acceptance criterion
- A live-browser run with screenshots
- A markdown execution report
- An Allure HTML dashboard
- A consolidated **Excel test-case document** (10-column standard format) refreshed on every run

---

2. How it works

Six stages, fully traceable on disk:

#	Stage	Input	Output
1	Story capture	URL + AC pasted into UI form	`user-stories/<ID>-<slug>.md`
2	Planning	Live page exploration via Playwright MCP	`specs/<ID>-<slug>-plan.md`
3	Exploratory test	Browser snapshot of the form / flow	List of locators, validation rules, success state
4	Generation	Plan + exploration findings	`pages/<slug>/<Page>.ts` (POM) + `tests/<slug>/*.spec.ts`
5	Execution	Playwright runs the specs (headed by default)	Pass/fail + screenshots + Allure JSON
6	Reporting	Run output + assertions	`reports/<ID>-<slug>.md` + `reports/Test-Cases.xlsx` + Allure HTML + Playwright HTML

Key design principles enforced by the pipeline:

Principle	What it prevents
Strict AC mapping — one spec per AC, asserting it directly	Tests that pass without verifying the AC
No false greens — unverifiable ACs use `test.fail()` with a reason	Suites that look green but skip real assertions
Idempotent generation — same story → reuse, never regenerate	Accidental overwrites of working code
POM convention — locators centralized in one file per page	One-file fixes when the UI changes
Reuse over recreate — existing code is extended, not duplicated	Parallel implementations of the same feature

---

3. Tools used

Category	Tool	Role
Excel reporting	ExcelJS	Generates the styled 10-column test-case workbook on every run
Browser automation	Playwright	Drives Chromium / Firefox / WebKit; runs the generated specs
Browser MCP server	Playwright MCP	Exposes browser actions (snapshot, click, fill, evaluate) as tools so an LLM can explore the app live
Test framework	`@playwright/test`	`test()`, `expect()`, fixtures, projects
Test reporting	Allure + Playwright HTML reporter	Rich HTML dashboards with steps, screenshots, traces
UI server	Express	Hosts the visual test runner; NDJSON streaming for live logs
Test authoring (optional)	Claude Code (subscription)	Drives the `playwright-test-planner`, `playwright-test-generator`, and `playwright-test-healer` agents in `.claude/agents/`
CI	GitHub Actions	Runs the suite on push/PR, uploads four artifact bundles
Page Object Model	Custom abstract `BasePage` class	Shared navigation, screenshotting, and assertions
Markdown rendering	`marked`	Renders the markdown execution reports inside the UI

Zero paid API dependencies. Test authoring uses your existing Claude Code subscription (no per-call billing), or it can be done entirely by hand following the POM conventions.

---

4. Practical example — End-to-end run on moontower.aiimone.com

Scenario: Two real features tested against a live SaaS application.

Inputs (pasted into the UI form)

Feature 1 — User Signup

```
Application URL: https://moontower.aiimone.com/signup
Story ID:        SignUp-001
Story Title:     User Signup
Credentials:     email: arslan.moon@yopmail.com
Acceptance Criteria:
  AC1: Visit the site, fill the signup form, and register as a new user
  AC2: Verify the user is registered
```

Feature 2 — Login

```
Application URL: https://moontower.aiimone.com/Login
Story ID:        Login
Story Title:     Login User
Credentials:     email: developers@moontower.com
                 password: 12345678
Acceptance Criteria:
  AC1: Visit the site and try to login
```

What the pipeline produced (committed to the repo)

File	Lines	Purpose
`user-stories/SignUp-001-user-signup.md`	39	Formalized story
`user-stories/Login-login-user.md`	32	Formalized story
`specs/SignUp-001-user-signup-plan.md`	38	Test plan with field map + success criteria
`specs/Login-login-user-plan.md`	26	Test plan
`pages/user-signup/SignupPage.ts`	78	POM with step-1 + step-2 actions
`pages/login-user/LoginPage.ts`	34	POM with `login()` action
`tests/user-signup/fill-and-submit.spec.ts`	47	AC1 spec
`tests/user-signup/verify-registered.spec.ts`	45	AC2 spec
`tests/login-user/login-success.spec.ts`	24	AC1 spec
`reports/SignUp-001-user-signup.md`	67	Execution report
`reports/Login-login-user.md`	56	Execution report

Execution results (Chromium)

Suite	ACs	Result	Duration
`tests/user-signup` (SignUp-001)	2/2	✅ PASS	5.0 s
`tests/login-user` (Login)	1/1	✅ PASS	5.1 s
Total	**3/3**	✅ **PASS**	**10.1 s**

Deliverables produced by the run

- **Markdown reports**: `reports/Login-login-user.md`, `reports/SignUp-001-user-signup.md` — dynamically refreshed on every run (hand-written context preserved outside `<!-- agentic-qa:auto-start --> ... <!-- :auto-end -->` markers).
- **Excel workbook**: `reports/Test-Cases.xlsx` — 3 sheets (Summary + one per feature), 10 columns per row matching the standard QA template (TEST CASE ID / TEST SCENARIO / TEST CASE / PRE-CONDITION / TEST STEPS / TEST DATA / EXPECTED RESULT / POST CONDITION / ACTUAL RESULT / STATUS), styled headers, color-coded status cells.

Non-obvious findings the pipeline surfaced

During exploratory testing, the pipeline discovered three production behaviors the requirements did **not** spell out — each would have caused a flaky test if missed:

1. **Three fields on the signup form are `readonly`** despite appearing as plain textboxes — `Subdomain` (auto-derived from Restaurant Name), `Location Name`, `Address`. The POM correctly skips them.
2. **The signup form has two steps**, not one — Restaurant Info → password creation → submit. The pipeline handled the multi-step flow without manual hinting.
3. **Email uniqueness is server-enforced.** First run created the account; subsequent runs failed silently. The pipeline switched to yopmail `+tag` aliases (`arslan.moon+ac1<timestamp>@yopmail.com`) so every run gets a fresh email while still landing in the same inbox.

Concrete code sample — POM that came out of the pipeline

```
// pages/login-user/LoginPage.ts (excerpt)
export class LoginPage extends BasePage {
  readonly url = 'https://moontower.aiimone.com/Login';
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly signInButton: Locator;

  constructor(page: Page) {
    super(page);
    this.emailInput = page.getByRole('textbox', { name: 'Email' });
    this.passwordInput = page.getByRole('textbox', { name: 'Password' });
    this.signInButton = page.getByRole('button', { name: 'Sign In' });
  }

  async login(email: string, password: string): Promise<void> {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.signInButton.click();
  }
}
```

```
// tests/login-user/login-success.spec.ts (excerpt)
test('AC1 - valid credentials authenticate the user and route them to /select-location', async ({
  const login = new LoginPage(page);
  await login.goto();
  await login.expectLoaded();
  await login.login(EMAIL, PASSWORD);

  // Three independent post-login signals - all must hold:
  await expect(page).toHaveURL(/\/select-location$/, { timeout: 15_000 });
  await expect(page.getByRole('heading', { name: 'Select Your Location' })).toBeVisible();
  await expect(page.getByText(/^Restaurant:\s+/)).toBeVisible();
});
```

The test passes only if **all three** post-auth signals hold — URL match, heading visible, and a `Restaurant:` paragraph confirming an account context loaded.

---

5. How to reproduce / evaluate

```
# Clone + install
git clone <repo-url>
cd Agentic_QA_Automation_Pipeline
npm run setup            # installs deps + Playwright browsers

# Launch the visual test runner
npm run ui               # → http://localhost:3001
                         # → pick "user-signup" or "login-user" → ▶ Run Tests
```

Or run from the command line:

```
npx playwright test tests/user-signup --project=chromium
npx playwright test tests/login-user --project=chromium
```

Reports are written to:

- `reports/SignUp-001-user-signup.md` — markdown execution summary
- `reports/Login-login-user.md` — markdown execution summary
- `playwright-report/index.html` — Playwright's native HTML report
- `allure-report/index.html` — Allure dashboard (requires Java locally; CI generates it automatically)

---

6. Project status

- ✅ MVP scope complete (story → tests → execution → report)
- ✅ POM convention enforced across every generated feature
- ✅ CI workflow runs the suite on every push/PR
- ✅ Two real features verified end-to-end on a live SaaS app
- ✅ Framework requires zero paid API keys to operate
- ✅ Distributable as an npm package (`npx agentic-qa init` slots it into any existing Playwright repo)

Repository: Agentic_QA_Automation_Pipeline — full source, agent definitions, and reusable Claude Code prompts in QAEnd2EndPromptFile.md.